



VIT[®]
BHOPAL

Name: Lakshita joshi

Registration number: 26BAS10034

Course: B.Tech in Aerospace Engineering

School: SASL(school of applied science and language)

Faculty name: Ashfaq ahmad najar

1. Introduction

The Personal Expense Tracker is a simple Python-based application developed to help users record and manage their daily expenses. The program provides a menu-driven interface through which users can add expenses, view recorded expenses, and calculate the total amount spent.

The project is developed using basic Python programming concepts such as variables, lists, functions, loops, conditional statements, input and output, and arithmetic operators.

The main purpose of this project is to demonstrate how basic programming concepts can be combined to solve a simple real-life problem.

2. Problem Statement

People often spend money on different things such as food, transportation, shopping, books, and other daily requirements. When these expenses are recorded manually, it can become difficult to calculate the total amount spent.

The problem is to develop a simple program that allows a user to record different expenses and calculate the total amount spent.

The proposed Expense Tracker solves this problem by providing a simple menu-driven Python program for recording and managing expenses.

3. Objectives

The main objectives of the Expense Tracker are:

1. To create a simple program for recording daily expenses.
2. To allow users to add multiple expenses.
3. To display all recorded expenses.
4. To calculate the total amount spent.
5. To provide a simple and user-friendly menu.
6. To apply basic Python programming concepts.
7. To understand how a real-life problem can be solved using programming.

4. Functional Requirements

The system should provide the following functions:

4.1 Add Expense

The user can enter the name and amount of an expense. The program stores the expense in a list.

4.2 View Expenses

The user can view all the expenses that have been recorded during the current program session.

4.3 Calculate Total Expense

The program calculates the sum of all recorded expenses and displays the total amount.

4.4 Exit

The user can exit the program by selecting the Exit option from the menu.

4.5 Invalid Input Handling

If the user enters an invalid menu option, the program displays an appropriate message and shows the menu again.

5. Non-Functional Requirements

5.1 Simplicity

The program should be simple and easy to understand for a beginner.

5.2 Usability

The menu should be clear so that the user can easily select the required option.

5.3 Performance

The program should perform basic calculations quickly for the number of expenses entered by the user.

5.4 Reliability

The program should correctly calculate and display the recorded expenses and their total.

5.5 Maintainability

The program should be organized into separate functions so that it can be easily modified or improved.

5.6 Compatibility

The program should run on a computer with Python installed.

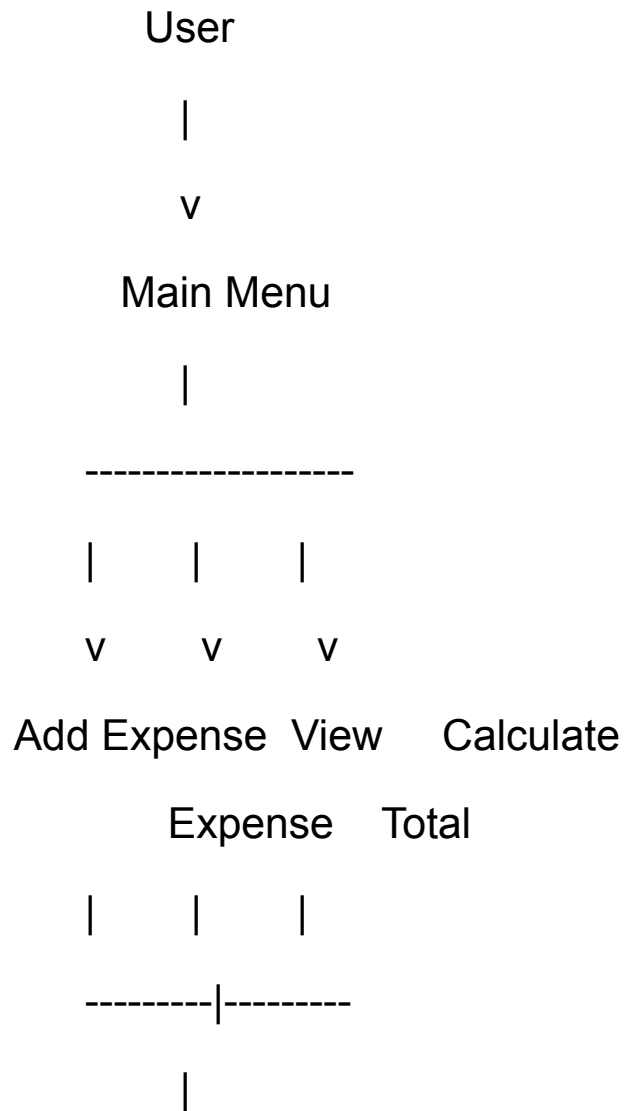
6. System Architecture

The Expense Tracker uses a simple program-based architecture.

The system consists of the following main components:

1. User
2. Main Menu
3. Expense Storage
4. Add Expense Function
5. View Expense Function
6. Total Expense Function

Architecture



Expense List

The user interacts with the main menu. Based on the selected option, the appropriate function is executed. The expenses are stored in a Python list.

7. Workflow

The working process of the Expense Tracker is as follows:

1. Start the program.
2. Display the main menu.
3. Ask the user to select an option.
4. If the user selects Add Expense, ask for the expense name and amount.
5. Store the expense in the list.
6. If the user selects View Expenses, display all recorded expenses.
7. If the user selects Total Expense, calculate and display the total.
8. If the user selects Exit, terminate the program.
9. If an invalid option is entered, display an error message.
10. Return to the main menu until the user chooses Exit.

Workflow

Start

|

v

Display Menu

|

v

Enter Choice

|

+----- 1 -----> Add Expense

|

|

|

v

|

Store Expense

|

+----- 2 -----> View Expenses

|

+----- 3 -----> Calculate Total

|

+----- 4 -----> Exit

|

v

Return to Menu

8. Use Case

Actor

The main actor of the system is the **User**.

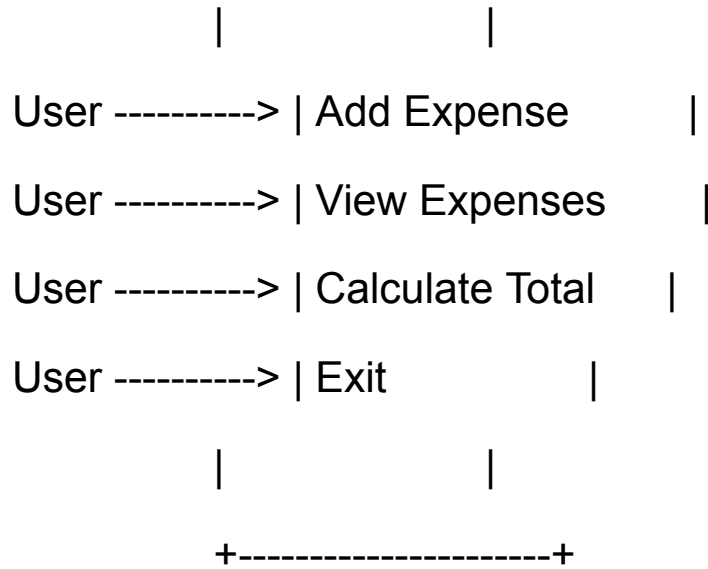
Use Cases

User Action	System Response
Add Expense	Stores the expense name and amount
View Expenses	Displays all recorded expenses
Calculate Total	Calculates and displays total expense
Exit	Closes the program
Invalid Choice	Displays an error message

Use Case Diagram

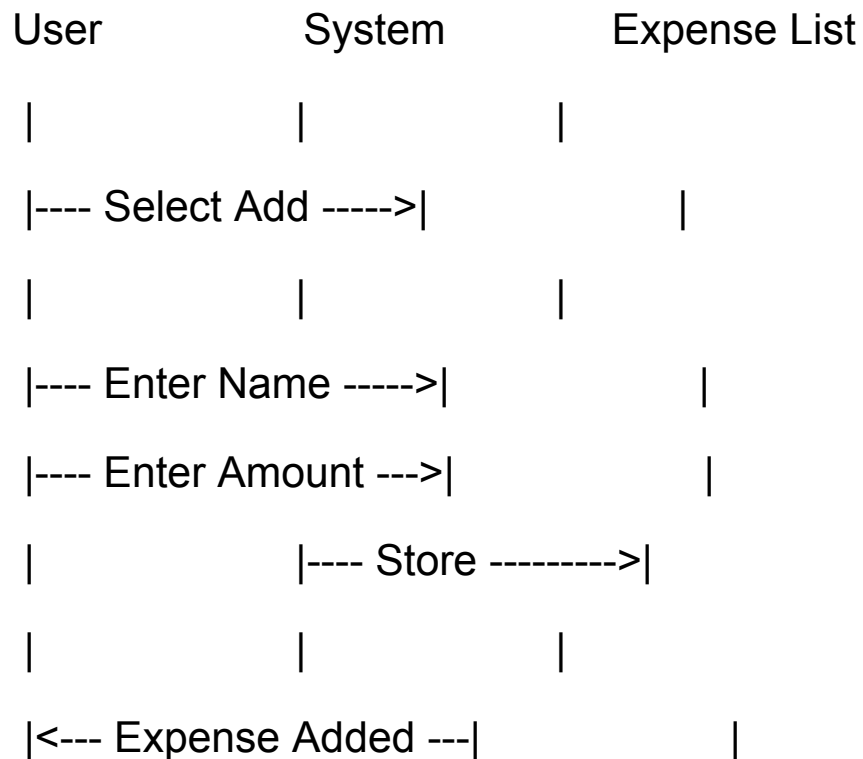
+-----+

| Expense Tracker |



9. Sequence Diagram

The sequence of interaction between the user and the system is shown below.



```

|           |           |
|---- Select View ---->|           |
|           |<---- Get Data -----|
|<----- Expenses ----|           |
|           |           |
|---- Select Total --->|           |
|           |<---- Get Data -----|
|<----- Total -----|           |
|           |           |

```

10. Component Design

The project is divided into simple components using Python functions.

10.1 Main Program

Controls the menu and takes input from the user.

10.2 Add Expense Function

The `add_expense()` function accepts the expense name and amount and stores them in the expense list.

10.3 View Expense Function

The `view_expenses()` function displays all recorded expenses.

10.4 Total Expense Function

The `total_expense()` function adds the amounts of all expenses and displays the total.

10.5 Expense List

A Python list is used to store the expenses.

Each expense is stored as:

`[expense_name, amount]`

11. Design Decisions

The following design decisions were made while developing the project:

11.1 Python

Python was selected because it is simple, easy to understand, and suitable for beginner-level programming.

11.2 List for Storage

A list was selected to store expenses because lists are part of the Python syllabus and can store multiple values.

11.3 Functions

Separate functions were created for different operations. This makes the program easier to understand and maintain.

11.4 Menu-Driven Design

A menu-driven approach was selected because it provides a simple way for the user to interact with the program.

11.5 No Database

A database was not used because the project is designed as a basic first-semester Python project. The expenses are stored only while the program is running.

12. Implementation

The Expense Tracker was implemented using Python.

The program uses a list called `expenses` to store the recorded expenses.

The main functions are:

`add_expense()`

`view_expenses()`

`total_expense()`

The `while` loop keeps the menu running until the user selects the Exit option.

The `if-elif-else` statements are used to perform different operations based on the user's choice.

The `for` loop is used to display expenses and calculate the total amount.

The main implementation uses basic Python concepts without external libraries.

13. Sample Result

Main Menu

Expense Tracker

1. Add Expense
2. View Expenses
3. Total Expense
4. Exit

Enter your choice:

Adding Expense

Enter your choice: 1

Enter expense name: Food

Enter amount: 150

Expense added

Adding Another Expense

Enter your choice: 1

Enter expense name: Bus

Enter amount: 50

Expense added

Viewing Expenses

Enter your choice: 2

Expenses:

Food - 150.0

Bus - 50.0

Calculating Total

Enter your choice: 3

Total expense = 200.0

The sample results show that the program successfully adds expenses, displays them, and calculates the total amount.

14. Testing

Testing was performed to check whether the different functions of the Expense Tracker work correctly.

Test Case	Input	Expected Result
TC01	Add Food = 150	Expense added
TC02	Add Bus = 50	Expense added
TC03	Select View Expenses	Food and Bus are displayed
TC04	Select Total Expense	Total = 200
TC05	View without expenses	No expenses found
TC06	Total without expenses	Total = 0
TC07	Select Exit	Program terminates

TC08	Enter invalid option	Invalid choice displayed
------	----------------------	--------------------------

The test cases confirm that the main functions of the program work as expected.

15. Challenges Faced

During the development of the project, the following challenges were faced:

1. Understanding how to store multiple expenses in a list.
2. Understanding how functions work in Python.
3. Using loops to display multiple expenses.
4. Calculating the total amount correctly.
5. Designing a simple menu-driven program.
6. Handling invalid menu choices.

These challenges helped in understanding the practical use of basic Python programming concepts.

16. Learning Outcomes

Through this project, I learned:

1. How to create a Python program to solve a real-life problem.
2. How to use lists for storing multiple values.
3. How to create and use functions.
4. How to use loops and conditional statements.
5. How to take input from the user.

6. How to perform calculations using Python.
7. How to test a program using different test cases.
8. How to organize a project and document its functionality.
9. How to upload and manage project files on GitHub.

17. Future Enhancements

The project can be improved in the future by adding:

1. Permanent storage using files.
2. Expense categories such as Food, Travel, Shopping, and Education.
3. Monthly and yearly expense reports.
4. Search and delete expense options.
5. A graphical user interface.
6. Charts and graphs for expense analysis.
7. User login and account management.
8. Database support for permanent storage.

18. Conclusion

The Personal Expense Tracker is a simple Python application developed to record and calculate daily expenses. The project successfully demonstrates basic programming concepts such as variables, lists, functions, loops, conditional statements, and arithmetic operations.

The project provides a simple menu-driven interface that allows users to add expenses, view recorded expenses, and calculate the total amount spent.

This project helped in understanding how programming concepts can be combined to solve a practical problem. It also provided practical experience in program development, testing, documentation, and GitHub project management.

19. References

1. Python Software Foundation. Python Documentation.
2. Course material for CSE1021: Introduction to Problem Solving and Programming.
3. Class notes and lectures on Python programming.
4. GitHub Documentation for repository and file management.